

React 1 – 8 HandsOns

INTRODUCTION TO THE REACT SPA and Its Benefits

What is a SPA?

A **Single-Page Application (SPA)** is a type of web application that loads a single HTML page and dynamically updates the content as the user interacts with the app. Instead of loading entire new pages from the server, SPA fetches only data and updates the view using JavaScript.

Benefits of SPA:

- **Fast Navigation:** Only parts of the page are updated; no full-page reloads.
- **Improved User Experience:** Smooth and seamless transitions between views.
- **Efficient Data Handling:** Uses AJAX or Fetch API to load data without page reload.
- **Reduced Server Load:** Server only sends data, not entire HTML files.
- **Easier Frontend-Backend Separation:** Backend provides APIs; frontend handles UI.

Define React and Identify Its Working

What is React?

React is an open-source **JavaScript library** used for building **user interfaces**, especially for Single-Page Applications. It was developed by Facebook and is widely used in frontend development.

How React Works:

- **Component-Based:** UI is broken into small, reusable pieces called components.
- **JSX Syntax:** Combines HTML-like syntax with JavaScript logic.
- **Virtual DOM:** Efficiently updates only the parts of the DOM that changed.
- **Unidirectional Data Flow:** Data flows from parent to child using props.
- **Declarative UI:** Developers describe what the UI should look like, and React handles the rendering.

SPA vs MPA (Multi-Page Application)

Comparison Between SPA and MPA:

Feature	SPA	MPA
---------	-----	-----

Loading	Loads once, then updates via JS Reloads full page on each action	
Speed	Faster after initial load	Slower due to full reloads
Server Interaction	Uses APIs, minimal HTML reload	Full page from server per request
SEO	Requires additional handling	Better SEO support by default
User Experience	Smooth, app-like	Can feel slower, less fluid
Technologies	React, Angular, Vue, etc.	Traditional backend frameworks

Pros & Cons of SPA

Pros of SPA:

- **Fast Performance** after initial load
- **Seamless User Interaction**
- **Reusability** of components
- **Works well with mobile apps**
- **Frontend and Backend can be developed**

separately Cons of SPA:

- **SEO Challenges** (content may not be indexed properly)
- **Initial Load Time** can be large
- **Browser History Management** is complex
- **Security Concerns** if not implemented properly
- **JavaScript Dependency** (won't work if JS fails)

About React

React is a **JavaScript library for building user interfaces**, particularly for Single-Page Applications where content updates dynamically. It helps developers create large web applications that can change data, without reloading the page.

Key Points:

- Developed by **Facebook** in 2013

- Uses **components** to build UIs
- Efficient with **Virtual DOM**
- Written using **JSX**
- Often used with **Redux**, **React Router**, and other libraries

Use Cases:

- Dynamic websites
- Dashboards
- Social media platforms
- E-commerce frontends

Define Virtual DOM

What is Virtual DOM?

The **Virtual DOM** is a programming concept used by React to improve performance. It's a **lightweight copy of the actual DOM**, stored in memory. React uses it to track changes and update the UI efficiently.

How It Works:

1. React builds a virtual DOM tree.
2. When the state changes, React creates a new virtual DOM.
3. It compares the new and old trees (**diffing**).
4. It updates only the parts of the real DOM that changed (**reconciliation**).

Benefits:

- Faster updates
- Improved performance
- Less manipulation of the real DOM

Features of React

Key Features:

1. **Component-Based**: Build reusable and independent UI components.
2. **JSX (JavaScript XML)**: HTML-like syntax in JavaScript.

3. **Virtual DOM:** Efficient rendering using diff and reconciliation.
 4. **Unidirectional Data Flow:** Props and state control data flow.
 5. **Declarative UI:** Describe what the UI should look like.
 6. **Hooks:** Functional components can manage state and lifecycle.
 7. **Rich Ecosystem:** React Router, Redux, Next.js, etc.
 8. **Developer Tools:** Great tools for debugging and performance analysis.
-

1. React Components

Components are the core of React. They are JavaScript functions or classes that return **JSX** (JavaScript XML), which tells React what to render on the screen.

2. Components vs JavaScript Functions

- Both are functions, but components return **UI elements** using JSX.
- Components have additional features like **props**, **state**, and **lifecycle methods**, unlike regular JS functions.

3. Types of Components

- **Function Components** – Simpler and modern; use hooks like `useState`.
- **Class Components** – Use lifecycle methods and `this.state`.

4. Class Component

- A component created using ES6 class syntax.
- Must have a `render()` method.
- Example:

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

5. Function Component

- Simpler component written as a function.
- Can use hooks like `useState`, `useEffect`.

- Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

1. **React Components**

React encourages breaking the UI into independent, reusable pieces (components).

2. **Components vs JavaScript Functions**

JS functions do logic; React components return UI and support interactivity.

3. **Types of Components**

- **Presentational** – Focus on UI only.
- **Container** – Manage data, state, and logic.

4. **Class Component**

- More powerful; has lifecycle methods and state management using `this.state`.

5. **Function Component**

- Preferred in modern React due to simplicity and use of **hooks**.

6. **Component Constructor**

- Called before the component mounts.
- Used to initialize state and bind event handlers.

```
constructor(props) {  
  super(props);  
  this.state = { count: 0 };  
}
```

7. **Render() Function**

- Mandatory in class components.
 - Returns JSX to display UI on the screen.
-

1. Need & Benefits of Lifecycle

Lifecycle methods help you **control what happens** when a component is created, updated, or destroyed — useful for tasks like fetching data or cleaning up timers.

2. Lifecycle Hook Methods

- `constructor()` – Initialization
- `componentDidMount()` – Called after component is added to the DOM
- `componentDidUpdate()` – Called after re-rendering
- `componentWillUnmount()` – Cleanup before removal

3. Rendering Sequence

1. `constructor()`
 2. `render()`
 3. `componentDidMount()`
 4. On updates: `render()` → `componentDidUpdate()`
 5. On removal: `componentWillUnmount()`
-

1. Need for Styling

Styling helps in creating visually appealing and responsive UIs that improve user experience.

2. CSS Modules

- Local scoped CSS files.
- Prevent conflicts by generating unique class names.
- Example:

```
import styles from './App.module.css';  
  
<div className={styles.title}>Hello</div>
```

3. Inline Styles

- Write styles directly in the JSX using a JavaScript object.
- Example:

```
<div style={{ color: 'red', fontSize: '20px' }}>Hi</div>
```

1. React Router – Need & Benefits

React Router allows SPA (Single Page App) behavior where navigation happens without reloading the page.

2. Router Components

- `<BrowserRouter>` – Main wrapper for routing
- `<Route>` – Defines a route path and component
- `<Link>` – Used for navigation
- `<Navigate>` – Redirects the user

3. Types of Routers

- **BrowserRouter** – Uses HTML5 history API
- **HashRouter** – Uses URL hash (#)

4. URL Parameter Passing

- Example: `/user/:id`
- Accessed using `useParams()` hook in function components

5. Props

- Short for properties; passed from parent to child components.
- Read-only and help customize components.

6. Default Props

- Provide fallback values when no props are supplied.
- Example:

```
MyComponent.defaultProps = {  
  name: 'Guest'  
};
```

7. State vs Props

- **State** is internal and can change.
- **Props** are external and immutable.

8. `ReactDOM.render()`

- Renders a React component to the DOM.
- Example:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

1. React State

- An object that holds **dynamic data** inside a component.
- Used to re-render UI when data changes.
- In function components:

```
const [count, setCount] = useState(0);
```
- In class components:

```
this.state = { count: 0 };
```